Programming Assignment 1

Trigram Language Model with Story Generation

Natural Language Processing — CSC4233

Total Marks: 15 | Bonus: +5 | Language: Python 3

---

1. Overview

In this assignment you will build a Trigram Language Model from scratch in Python. Your model will learn word patterns from a classic literature corpus and use them to generate a 30-word story starting from a seed phrase. You will also implement Laplace Smoothing to handle unseen word sequences and evaluate your model using Perplexity.

By completing this assignment you will:

- Understand how n-gram language models capture statistical patterns in text
- Implement Laplace Smoothing to solve the zero-probability problem
- Build a greedy text generator using conditional trigram probabilities
- Evaluate model quality with Perplexity on a held-out test sentence
- (Bonus) Explore Simple Linear Interpolation as an advanced smoothing method

2. Learning Objectives

1. Apply text preprocessing: lowercasing, punctuation removal, tokenization
2. Construct trigram frequency tables from tokenized corpora
3. Implement and apply Laplace Smoothing correctly
4. Build a deterministic (greedy) next-word predictor
5. Compute Perplexity on a held-out test sentence
6. (Bonus) Implement and compare Simple Linear Interpolation

3. Dataset

Use the NLTK Gutenberg corpus. The recommended file is Shakespeare's Julius Caesar, but you may use any file from the corpus or supply your own .txt file.

Option A — NLTK Gutenberg (recommended)

```
import nltk
nltk.download('gutenberg')
from nltk.corpus import gutenberg
raw_text = gutenberg.raw('shakespeare-caesar.txt')
```

Option B — Custom .txt File

Any plain-text English file with at least 5,000 words. Include the file in your submission zip and load it with:

```
with open('your_corpus.txt', 'r', encoding='utf-8') as f:
    raw_text = f.read()
```

4. Step-by-Step Instructions

Step 1 — Preprocessing (2 marks)

Clean the raw text before building the model. Your preprocessing must:

1. Convert all text to lowercase
2. Remove all punctuation (keep only letters and spaces)
3. Tokenize into a list of word strings

Required function signature:

```
def preprocess(text: str) -> list[str]:
    """Returns lowercase word tokens with no punctuation."""
```

Example:

```
Input:  "Friends, Romans, Countrymen!"
Output: ['friends', 'romans', 'countrymen']
```

Step 2 — Build the Trigram Model (3 marks)

A trigram model captures P(word | two preceding words). Build a frequency table mapping each ($word_1$, $word_2$) context to the words that follow it and their counts.

Required function signature:

```
def build_trigram_model(tokens: list[str]) -> dict:
    """Returns trigram_counts[(w1,w2)][next_word] = count."""
```

Partial example of expected output:

```
{ ('the', 'king'): {'is': 3, 'of': 5, 'shall': 1}, ... }
```

Hint: Use collections.defaultdict(Counter) for clean counting.

Step 3 — Laplace Smoothing (3 marks)

Add-1 smoothing prevents zero probabilities for unseen trigrams. The formula is:

```
P(w | context) = (count(context, w) + 1) / (count(context) + V)
```

where V is the total vocabulary size (number of unique word types).

Required function signature:

```
def laplace_smoothing(trigram_counts: dict, vocab_size: int) -> dict:
    """Returns smoothed_probs[(w1,w2)][next_word] = probability."""
```

Step 4 — Text Generation (4 marks)

Generate a 30-word story from a two-word seed phrase using your smoothed trigram model. At each step, greedily pick the next word with the highest probability.

Algorithm:

4. Start with seed = ['the', 'king'] (or any two-word phrase of your choice)
5. Look up the current last two words as the context
6. Select the next word with the highest smoothed probability
7. Append the word and repeat until you have 30 words total

Required function signature:

```
def generate_text(seed: list[str], smoothed_probs: dict,
                  vocab: list[str], num_words: int = 30) -> str:
    """Returns a generated story string of num_words words."""
```

Note: If the seed context is not in the model, select a random word from the vocabulary.

Step 5 — Perplexity Evaluation (3 marks)

Perplexity measures how well your model predicts unseen text. Lower = better.

$$PP = \exp(-1/N \times \Sigma \log P(w_i \mid w_{i-2}, w_{i-1}))$$

Use your Laplace-smoothed probabilities and natural log (math.log). Apply the formula over all trigrams in the test sentence.

Required function signature:

```
def compute_perplexity(test_tokens: list[str], smoothed_probs: dict,
                       vocab_size: int) -> float:
    """Returns perplexity as a float."""
```

Evaluate on the test sentence: "the king is dead" and print the result.

5. Bonus Challenge — Simple Linear Interpolation (+5 marks)

Replace Laplace Smoothing with Simple Linear Interpolation (SLI), which combines unigram, bigram, and trigram probabilities using mixing weights that sum to 1.

```
P_SLI(w | w₁,w₂) = λ₃·P(w|w₁,w₂) + λ₂·P(w|w₂) + λ₁·P(w)
```

Suggested weights: $\lambda_3 = 0.6$, $\lambda_2 = 0.3$, $\lambda_1 = 0.1$. You are free to tune them.

To earn all 5 bonus marks you must:

1. Implement build_unigram_model() and build_bigram_model()
2. Implement interpolated_probability() using all three models
3. Re-generate a 30-word story using interpolated probabilities
4. Compute perplexity for both Laplace and Interpolation versions
5. Write 3–5 sentences comparing the two generated stories and perplexity values

6. Submission Requirements

File Structure

Submit a single .zip file named: StudentID_Section_Name.zip

```
StudentID_Section_Name /
├── ngram_model.py      ← your main Python file (required)
├── output.txt          ← printed results copied from terminal (required)
└── corpus.txt          ← only if using a custom dataset
```

Required Output (output.txt)

Your script must print the following when run with: python ngram_model.py

```
=== Preprocessing ===
Vocabulary size: <N>
Total tokens:    <N>

=== Text Generation (Laplace Smoothing) ===
Seed: the king
Generated: <your 30-word story here>

=== Perplexity ===
Test sentence: 'the king is dead'
Perplexity: <value>

=== Bonus (if attempted) ===
Generated (Interpolation): <30-word story>
Perplexity (Interpolation): <value>
```

```
Comparison: <3-5 sentence comparison>
```

Coding Rules

- All code in a single file: ngram_model.py
- Python 3.8 or later
- Allowed libraries only: nltk, re, math, numpy, collections, random, string
- All 5 required functions must exist with the exact signatures shown above
- Do NOT use any n-gram library for the model (e.g. do not call nltk.ngrams as your implementation)
- Code must run without errors: python ngram_model.py

7. Grading Rubric

Component	What We Check	Marks
Dataset & Preprocessing	Lowercase, no punctuation, correct tokenization using NLTK	2
Trigram Model	Correct trigram frequency counting from corpus tokens	3
Laplace Smoothing	Correct formula: (count + 1) / (context_total + V)	3
Text Generation	30-word story from seed, greedy highest-probability selection	4
Perplexity Evaluation	Correct formula applied to test sentence, valid numeric output	3
TOTAL		**15**

Bonus Marks

Bonus Task	Criteria	Marks
build_unigram_model() & build_bigram_model()	Both unigram and bigram models built correctly	2
interpolated_probability()	SLI formula applied with $\lambda_1+\lambda_2+\lambda_3 = 1$	2
Comparison paragraph	3–5 sentences comparing Laplace vs Interpolation output/PPL	1
BONUS TOTAL		**+5**

8. Penalties

Issue	Description	Penalty
Code does not run	Syntax or import errors prevent execution	−5 (max)
Missing function	Per missing required function (up to −3 total)	−1 each
Late submission	Per calendar day past the deadline	−2/day
Plagiarism	Zero tolerance — academic integrity policy	0 + report to OSA

9. Tips & Common Mistakes

- Start with preprocess() and verify output before building the model.
- Use collections.defaultdict(lambda: defaultdict(int)) for your trigram table.
- Always handle the edge case where the context tuple is not in the model.
- For perplexity: if a probability is somehow zero, add a small epsilon (1e-10) to avoid log(0).
- Test each function individually — do not wait until the end to run everything together.
- The 30-word count includes the two seed words.

10. Academic Integrity

All submitted work must be entirely your own. You may discuss concepts with peers but must not share or copy code. Submissions are checked for similarity. Any violation results in a zero mark and a formal report to the Office of Student Affairs. Use of AI-generated code is not permitted.

Submission & Deadline

Upload your .zip via the course portal before the stated deadline.

Deadline: 28th March 2026, Saturday 11.45PM

Late submission is not accepted. Post questions on the course TEAMS or attend office hours.

Good luck!